C++ Chapter 03 Notes

Chapter 03 — Notes

A Strategy for Debugging

When debugging a program, in most cases the vast majority of your time will be spent trying to find where the error actually is. Once the issue is found, the remaining steps (fixing the issue and validating that the issue was fixed) are often trivial in comparison.

In this lesson, we'll start exploring how to find errors.

Finding problems via code inspection

Let's say you've noticed a problem, and you want to track the cause of that specific problem down. In many cases (especially in smaller programs), we can approximate where the issue is likely to be based on the nature of the error and the way the program is structured.

Consider the following program snippet:

```
int main()
{
    getNames(); // ask user to enter a bunch of names
    sortNames(); // sort them in alphabetical order
    printNames(); // print the sorted list of names

    return 0;
}
```

If you expected this program to print the names in alphabetical order, but it printed them in opposite order instead, the problem is probably in the *sortNames* function. In cases where you can narrow the problem down to a specific function, you may be able to spot the issue just by looking at the code.

However, as programs get more complex, finding issues by code inspection becomes more complex as well.

First, there's a lot more code to look at. Looking at every line of code in a program that is thousands of lines long can take a really long time (not to mention it's incredibly boring). Second, the code itself tends to be more complex, with more possible places for things to go wrong. Third, the code's behavior may not give you many clues as to where things are going wrong. If you wrote a program to output stock recommendations and it actually output nothing at all, you probably wouldn't have much of a lead on where to start looking for the problem.

Finally, bugs can be caused by making bad assumptions. It's almost impossible to visually spot a bug caused by a bad assumption, because you're likely to make the same bad assumption when inspecting the code, and not notice the error. So if we have an issue that we can't find via code inspection, how do we find it?

Finding problems by running the program

Fortunately, if we can't find an issue via code inspection, there is another avenue we can take: we can watch the behavior of the program as it runs, and try to diagnose the issue from that. This approach can be generalized as:

1. Figure out how to reproduce the problem
2. Run the program and gather information to narrow down where the problem is
3. Repeat the prior step until you find the problem

For the rest of this chapter, we'll discuss techniques to facilitate this approach.

Reproducing the problem

The first and most important step in finding a problem is to be able to *reproduce the problem*. Reproducing the problem means making the problem appear in a consistent manner. The reason is simple: it's extremely hard to find an issue unless you can observe it occurring.

Back to our ice dispenser analogy – let's say one day your friend tells you that your ice dispenser isn't working. You go to look at it, and it works fine. How would you diagnose the problem? It would be very difficult. However, if you could actually see the issue of the ice dispenser not working, then you could start to diagnose why it wasn't working much more effectively.

If a software issue is blatant (e.g. the program crashes in the same place every time you run it) then reproducing the problem can be trivial. However, sometimes reproducing an issue can be a lot more difficult. The problem may only occur on certain computers, or in particular circumstances (e.g. when the user enters certain input). In such cases, generating a set of reproduction steps can be helpful.
Reproduction steps are a list of clear and precise steps that can be followed to cause an issue to recur with a high level of predictability. The goal is to be able to cause the issue to reoccur as much as possible, so we can run our program over and over and look for clues to determine what's causing the problem. If the issue can be reproduced 100% of the time, that's ideal, but less than 100% reproducibility can be okay. An issue that occurs only 50% of the time simply means it'll take twice as long to diagnose the issue, as half the time the program won't exhibit the problem and thus not contribute any useful diagnostic information.

Homing in on issues

Once we can reasonably reproduce the problem, the next step is to figure out where in the code the problem is. Based on the nature of the problem, this may be easy or difficult. For the sake of example, let's say we don't have much of an idea where the problem actually is. How do we find it?

An analogy will serve us well here. Let's play a game of hi-lo. I'm going to ask you to guess a number between 1 and 10. For each guess you make, I'll tell you whether each guess is too high, too low, or correct. An instance of this game might look like this:

```
You: 5
Me: Too low
You: 8
Me: Too high
You: 6
Me: Too low
You: 7
Me: Correct
```

In the above game, you don't have to guess every number to find the number I was thinking of. Through the process of making guesses and considering the information you learn from each guess, you can "home in" on the correct number with only a few guesses (if you use an optimal strategy, you can always find the number I'm thinking of in 4 or fewer guesses).

We can use a similar process to debug programs. In the worst case, we may have no idea where the bug is. However, we do know that the problem must be somewhere in the code that executes between the beginning of the program and the point where the program exhibits the first incorrect symptom that we can observe. That at least rules out the parts of the program that execute after the first observable symptom. But that still potentially leaves a lot of code to cover. To diagnose the issue, we'll make some educated guesses about where the problem is, with the goal of homing in on the problem quickly.

Often, whatever it was that caused us to notice the problem will give us an initial guess that's close to where the actual problem is. For example, if the program isn't writing data to a file when it should be, then the issue is probably somewhere in the code that handles writing to a file (duh!). Then we can use a hi-lo like strategy to try and isolate where the problem actually is.

For example:

- If at some point in our program, we can prove that the problem has not occurred yet, this is analogous to receiving a "too low" hi-lo result – we know the problem must be somewhere later in the program. For example, if our program is crashing in the same place every time, and we can prove that the program has not crashed at a particular point in the execution of the program, then the crash must be later in the code.
- If at some point in our program we can observe incorrect behavior related to the problem, then this is analogous to receiving a "too high" hi-lo result, and we know the problem must be somewhere earlier in the program. For example, let's say a program prints the value of some variable *x*. You were expecting it to print value *2*, but it printed *8* instead. Variable *x* must have the wrong value. If, at some point during the execution of our program, we can see that variable *x* already has value *8*, then we know the problem must have occurred before that point.

The hi-lo analogy isn't perfect – we can also sometimes remove entire sections of our code from consideration without gaining any information on whether the actual problem is before or after that point.

We'll show examples of all three of these cases in the next lesson.

Eventually, with enough guesses and some good technique, we can home in on the exact line causing the problem! If we've made any bad assumptions, this will help us discover where. When you've excluded everything else, the only thing left must be causing the problem. Then it's just a matter of understanding why.

What guessing strategy you want to use is up to you – the best one depends on what type of bug it is, so you'll likely want to try many different approaches to narrow down the issue. As you gain experience in debugging issues, your intuition will help guide you.

So how do we "make guesses"? There are many ways to do so. We're going to start with some simple approaches in the next chapter, and then we'll build on these and explore others in future chapters.

Basic Debugging Tactics

In the previous lesson, we explored a strategy for finding issues by running our programs and using guesswork to home in on where the problem is. In this lesson, we'll explore some basic tactics for actually making those guesses and collecting information to help find issues.

Debugging tactic #1: Commenting out your code

Let's start with an easy one. If your program is exhibiting erroneous behavior, one way to reduce the amount of code you have to search through is to comment some code out and see if the issue persists. If the issue remains unchanged, the commented out code probably wasn't responsible.

Consider the following code:

```
int main()
{
    getNames(); // ask user to enter a bunch of names
    doMaintenance(); // do some random stuff
    sortNames(); // sort them in alphabetical order
    printNames(); // print the sorted list of names

    return 0;
}
```

Let's say this program is supposed to print the names the user enters in alphabetical order, but its printing them in reverse alphabetical order. Where's the problem? Is *getNames* entering the names incorrectly? Is *sortNames* sorting them backwards? Is *printNames* printing them backwards? It could be any of those things. But we might suspect doMaintenance() has nothing to do with the problem, so let's comment it out.

```
int main()
{
    getNames(); // ask user to enter a bunch of names
//    doMaintenance(); // do some random stuff
    sortNames(); // sort them in alphabetical order
    printNames(); // print the sorted list of names

    return 0;
}
```

There are three likely outcomes:

- If the problem goes away, then *doMaintenance* must be causing the problem, and we should focus our attention there.
- If the problem is unchanged (which is more likely), then we can reasonably assume that *doMaintenance* wasn't at fault, and we can exclude the entire function from our search for now. This doesn't help us understand whether the actual problem is before or after the call to *doMaintenance*, but it reduces the amount of code we have to subsequently look through.
- If commenting out *doMaintenance* causes the problem to morph into some other related problem (e.g. the program stops printing names), then it's likely that *doMaintenance* is doing something useful that some other code is dependent on. In this case, we probably can't tell whether the issue is in *doMaintenance* or elsewhere, so we can uncomment *doMaintenance* and try some other approach.

> **Warning**
>
> Don't forget which functions you've commented out so you can uncomment them later!
>
> After making many debugging-related changes, it's very easy to miss undoing one or two. If that happens, you'll end up fixing one bug but introducing others!

> Having a good version control system is extremely useful here, as you can diff your code against the main branch to see all the debugging-related changes you've made (and ensure that they are reverted before you commit your change).

> **Tip**
>
> An alternate approach to repeatedly adding/removing or uncommenting/commenting debug statements is to use a 3rd party library that will let you leave debug statements in your code but compile them out in release mode via a preprocessor macro. dbg is one such header-only library that exists to help facilitate this (via the `DBG_MACRO_DISABLE` preprocessor macro).
>
> We discuss header-only libraries in lesson 7.9 – Inline functions and variables.

Debugging tactic #2: Validating your code flow

Another problem common in more complex programs is that the program is calling a function too many or too few times (including not at all).

In such cases, it can be helpful to place statements at the top of your functions to print the function's name. That way, when the program runs, you can see which functions are getting called.

> **Tip**
>
> When printing information for debugging purposes, use `std::cerr` instead of `std::cout`. One reason for this is that `std::cout` may be buffered, which means a bit of time may pass between when you ask `std::cout` to output text and when it actually does. If you output using `std::cout` and then your program crashes immediately afterward, `std::cout` may or may not have actually output yet. This can mislead you about where the issue is. On the other hand, `std::cerr` is unbuffered, which means anything you send to it will output immediately. This helps ensure all debug output appears as soon as possible (at the cost of some performance, which we usually don't care about when debugging).
>
> Using `std::cerr` also helps make clear that the information being output is for an error case rather than a normal case.
>
> We discuss when to use `std::cout` vs `std::cerr` further in lesson 9.4 – Detecting and handling errors.

Consider the following simple program that doesn't work correctly:

```
#include <iostream>

int getValue()
{
    return 4;
}

int main()
{
    std::cout << getValue << '\n';

    return 0;
}
```

You may need to disable "Treat warnings as errors" for the above to compile.

Although we expect this program to print the value *4*, it should print the value:

```
1
```

On Visual Studio (and possibly some other compilers), it may print the following instead:

```
00101424
```

Related content

We discuss why some compilers print `1` vs an address (and what to do if your compiler prints `1` but you want it to print an address) in lesson 20.1 – Function Pointers.

Let's add some debugging statements to these functions:

```
#include <iostream>

int getValue()
{
std::cerr << "getValue() called\n";
    return 4;
}

int main()
{
std::cerr << "main() called\n";
    std::cout << getValue << '\n';

    return 0;
}
```

Tip

When adding temporary debug statements, it can be helpful to not indent them. This makes them easier to find for removal later.

If you are using clang-format to format your code, it will try to auto-indent these lines. You can suppress the automatic formatting like this:

```
  // clang-format off
  std::cerr << "main() called\n";
  // clang-format on
```

Now when these functions execute, they'll output their names, indicating that they were called:

```
main() called
1
```

Now we can see that function *getValue* was never called. There must be some problem with the code that calls the function. Let's take a closer look at that line:

```
std::cout << getValue << '\n';
```

Oh, look, we forgot the parenthesis on the function call. It should be:

```
#include <iostream>

int getValue()
{
std::cerr << "getValue() called\n";
    return 4;
}

int main()
{
std::cerr << "main() called\n";
    std::cout << getValue() << '\n'; // added parenthesis here

    return 0;
}
```

This will now produce the correct output

```
main() called
getValue() called
4
```

And we can remove the temporary debugging statements.

Debugging tactic #3: Printing values

With some types of bugs, the program may be calculating or passing the wrong value.

We can also output the value of variables (including parameters) or expressions to ensure that they are correct.

Consider the following program that is supposed to add two numbers but doesn't work correctly:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

void printResult(int z)
{
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    int x{ getUserInput() };
    int y{ getUserInput() };

    int z{ add(x, 5) };
    printResult(z);

    return 0;
}
```

Here's some output from this program:

```
Enter a number: 4
Enter a number: 3
The answer is: 9
```

That's not right. Do you see the error? Even in this short program, it can be hard to spot. Let's add some code to debug our values:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

void printResult(int z)
{
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    int x{ getUserInput() };
std::cerr << "main::x = " << x << '\n';
    int y{ getUserInput() };
std::cerr << "main::y = " << y << '\n';

    int z{ add(x, 5) };
std::cerr << "main::z = " << z << '\n';
    printResult(z);

    return 0;
}
```

Here's the above output:

```
Enter a number: 4
main::x = 4
Enter a number: 3
main::y = 3
main::z = 9
The answer is: 9
```

Variables *x* and *y* are getting the right values, but variable *z* isn't. The issue must be between those two points, which makes function *add* a key suspect.

Let's modify function add:

```
#include <iostream>

int add(int x, int y)
{
std::cerr << "add() called (x=" << x <<", y=" << y << ")\n";
    return x + y;
}

void printResult(int z)
{
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    int x{ getUserInput() };
std::cerr << "main::x = " << x << '\n';
    int y{ getUserInput() };
std::cerr << "main::y = " << y << '\n';

    int z{ add(x, 5) };
std::cerr << "main::z = " << z << '\n';
    printResult(z);

    return 0;
}
```

Now we'll get the output:

```
Enter a number: 4
main::x = 4
Enter a number: 3
main::y = 3
add() called (x=4, y=5)
main::z = 9
The answer is: 9
```

Variable *y* had value 3, but somehow our function *add* got the value 5 for parameter *y*. We must have passed the wrong argument. Sure enough:

```
int z{ add(x, 5) };
```

There it is. We passed the literal *5* instead of the value of variable *y* as an argument. That's an easy fix, and then we can remove the debug statements.

One more example

This program is very similar to the prior one, but also doesn't work like it should:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

void printResult(int z)
{
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return --x;
}

int main()
{
    int x{ getUserInput() };
    int y{ getUserInput() };

    int z { add(x, y) };
    printResult(z);

    return 0;
}
```

If we run this code and see the following:

```
Enter a number: 4
Enter a number: 3
The answer is: 5
```

Hmmm, something is wrong. But where?

Let's instrument this code with some debugging:

```
#include <iostream>

int add(int x, int y)
{
std::cerr << "add() called (x=" << x << ", y=" << y << ")\n";
    return x + y;
}

void printResult(int z)
{
std::cerr << "printResult() called (z=" << z << ")\n";
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
std::cerr << "getUserInput() called\n";
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return --x;
}

int main()
{
std::cerr << "main() called\n";
    int x{ getUserInput() };
std::cerr << "main::x = " << x << '\n';
    int y{ getUserInput() };
std::cerr << "main::y = " << y << '\n';

    int z{ add(x, y) };
std::cerr << "main::z = " << z << '\n';
    printResult(z);

    return 0;
}
```

Now let's run the program again with the same inputs:

```
main() called
getUserInput() called
Enter a number: 4
main::x = 3
getUserInput() called
Enter a number: 3
main::y = 2
add() called (x=3, y=2)
main::z = 5
printResult() called (z=5)
The answer is: 5
```

Now we can immediately see something going wrong: The user is entering the value *4*, but main's *x* is getting value *3*. Something must be going wrong between where the user enters input and where that value is assigned to main's variable *x*. Let's make sure that the program is getting the correct value from the user by adding some debug code to function *getUserInput*:

```
#include <iostream>

int add(int x, int y)
{
std::cerr << "add() called (x=" << x << ", y=" << y << ")\n";
    return x + y;
}

void printResult(int z)
{
std::cerr << "printResult() called (z=" << z << ")\n";
    std::cout << "The answer is: " << z << '\n';
}

int getUserInput()
{
std::cerr << "getUserInput() called\n";
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
std::cerr << "getUserInput::x = " << x << '\n'; // added this additional line of debugging
    return --x;
}

int main()
{
std::cerr << "main() called\n";
    int x{ getUserInput() };
std::cerr << "main::x = " << x << '\n';
    int y{ getUserInput() };
std::cerr << "main::y = " << y << '\n';

    int z{ add(x, y) };
std::cerr << "main::z = " << z << '\n';
    printResult(z);

    return 0;
}
```

And the output:

```
main() called
getUserInput() called
Enter a number: 4
getUserInput::x = 4
main::x = 3
getUserInput() called
Enter a number: 3
getUserInput::x = 3
main::y = 2
add() called (x=3, y=2)
main::z = 5
printResult() called (z=5)
The answer is: 5
```

With this additional line of debugging, we can see that the user input is received correctly into getUserInput's variable *x*. And yet somehow main's variable *x* is getting the wrong value. The problem must be between those two points. The only culprit left is the return value from function *getUserInput*. Let's look at that line more closely.

```
return --x;
```

Hmmm, that's odd. What's that `--` symbol before x? We haven't covered that yet in these tutorials, so don't worry if you don't know what it means. But even without knowing what it means, through your debugging efforts, you can be reasonably sure that this particular line is at fault – and thus, it's likely this `--` symbol is causing the problem.

Since we really want *getUserInput* to return just the value of *x*, let's remove the `--` and see what happens:

```
#include <iostream>

int add(int x, int y)
{
std::

---

# Chapter Review

A **syntax error** is an error that occurs when you write a statement that is not valid according to the grammar of C++ language. The compiler will catch these.

A **semantic error** occurs when a statement is syntactically valid, but does not do what the programmer intended.

The process of finding and removing errors from a program is called **debugging**.

We can use a five step process to approach debugging:

1. Find the root cause.
2. Understand the problem.
3. Determine a fix.
4. Repair the issue.
5. Retest.

Finding an error is usually the hardest part of debugging.
```

```
        **Static analysis tools** are tools that analyze your code and look for semantic issues that may indicate problems w
your code.

        Being able to reliably reproduce an issue is the first and most important step in debugging.

        There are a number of tactics we can use to help find issues:

        - Commenting out code.
        - Using output statements to validate your code flow.
        - Printing values.

        When using print statements to debug, use *std::cerr* instead of *std::cout*. But even better, avoid debugging via p
statements.

        A **log file** is a file that records events that occur in a program. The process of writing information to a log fi
called **logging**.

        The process of restructuring your code without changing how it behaves is called **refactoring**. This is typically
to make your program more organized, modular, or performant.

        **Unit testing** is a software testing method by which small units of source code are tested to determine whether th
correct.

        **Defensive programming** is a technique whereby the programmer tries to anticipate all of the ways the software cou
misused. These misuses can often be detected and mitigated.

        All of the information tracked in a program (variable values, which functions have been called, the current point of
execution) is part of the **program state**.

        A **debugger** is a tool that allows the programmer to control how a program executes and examine the program state
the program is running. An **integrated debugger** is a debugger that integrates into the code editor.

        **Stepping** is the name for a set of related debugging features that allow you to step through your code statement
statement.

        **Step into** executes the next statement in the normal execution path of the program, and then pauses execution. If
statement contains a function call, *step into* causes the program to jump to the top of the function being called.

        **Step over** executes the next statement in the normal execution path of the program, and then pauses execution. If
statement contains a function call, *step over* executes the function and returns control to you after the function has
been executed.

        **Step out** executes all remaining code in the function currently being executed and then returns control to you wh
function has returned.

        **Run to cursor** executes the program until execution reaches the statement selected by your mouse cursor.

        **Continue** runs the program, until the program terminates or a breakpoint is hit. **Start** is the same as continu
just from the beginning of the program.

        A **breakpoint** is a special marker that tells the debugger to stop execution of the program when the breakpoint is
reached.
```

```
        The **set next statement** command allows us to change the point of execution to some other statement (sometimes
informally called jumping). This can be used to jump the point of execution forwards and skip some code that would
otherwise execute, or backwards and have something that already executed run again.

        **Watching a variable** allows you to inspect the value of a variable while the program is executing in debug mode.
**watch window** allows you to examine the value of variables or expressions.

        The **call stack** is a list of all the active functions that have been executed to get to the current point of exec
The **call stack window** is a debugger window that shows the call stack.

        ## Quiz time

        The following program is supposed to add two numbers, but doesn't work correctly.

        Use the integrated debugger to step through this program and watch the value of x. Based on the information you lear
the following program:

        ```cpp
        #include <iostream>

        int readNumber(int x)
        {
            std::cout << "Please enter a number: ";
            std::cin >> x;
            return x;
        }

        void writeAnswer(int x)
        {
            std::cout << "The sum is: " << x << '\n';
        }

        int main()
        {
            int x {};
            readNumber(x);
            x = x + readNumber(x);
            writeAnswer(x);

            return 0;
        }
```

Show Solution

The major issue here is in the second line of function *main* – the return value of readNumber isn't assigned to anything, so it is discarded. A minor issue is that *readNumber* is taking an argument when it should have a local variable instead.
```

```
#include <iostream>

int readNumber()
{
  std::cout << "Please enter a number: ";
  int x {};
  std::cin >> x;
  return x;
}

void writeAnswer(int x)
{
  std::cout << "The sum is: " << x << '\n';
}

int main()
{
  int x { readNumber() };
  x = x + readNumber();
  writeAnswer(x);

  return 0;
}
```

The following program is supposed to divide two numbers, but doesn't work correctly.

Use the integrated debugger to step through this program. For inputs, enter 8 and 4. Based on the information you learn, fix the following program:

```cpp
#include <iostream>

int readNumber()
{
    std::cout << "Please enter a number: ";
    int x {};
    std::cin >> x;
    return x;
}

void writeAnswer(int x)
{
    std::cout << "The quotient is: " << x << '\n';
}

int main()
{
    int x{ };
    int y{ };
    x = readNumber();
    x = readNumber();
    writeAnswer(x/y);

    return 0;
}
```

Show Solution

The issue here is that the second call to *readNumber* accidentally assigns its value to x instead of y, resulting in a division by 0, which causes undefined behavior.

```cpp
#include <iostream>

int readNumber()
{
  std::cout << "Please enter a number: ";
  int x {};
  std::cin >> x;
  return x;
}

void writeAnswer(int x)
{
  std::cout << "The quotient is: " << x << '\n';
}

int main()
{
  int x{ readNumber() };
  int y{ readNumber() };
  writeAnswer(x/y);

  return 0;
}
```

You may notice that when the second input doesn't divide evenly into the first, this program appears to produce an incorrect answer. When doing division with integers, C++ will drop any fractional parts of the quotient. We'll discuss this in more detail in lesson 6.2 – Arithmetic operators.

What does the call stack look like in the following program when the point of execution is on line 4? Only the function names are needed for this exercise, not the line numbers indicating the point of return.

We talk about the call stack in lesson 3.9 – Using an integrated debugger: The call stack.

```
#include <iostream>

void d()
{ // here
}

void c()
{
}

void b()
{
    c();
    d();
}

void a()
{
    b();
}

int main()
{
    a();

    return 0;
}
```

Show Solution

d b a main

Extra credit: The following program is supposed to add two numbers, but doesn't work correctly.

Use the integrated debugger to step through this program. For inputs, enter 8 and 4. Based on the information you learn, fix the following program:

```
#include <iostream>

int readNumber()
{
    std::cout << "Please enter a number: ";
    char x{};
    std::cin >> x;

    return x;
}

void writeAnswer(int x)
{
    std::cout << "The sum is: " << x << '\n';
}

int main()
{
    int x { readNumber() };
    int y { readNumber() };
    writeAnswer(x + y);

    return 0;
}
```

Show Solution

The issue is the `char` data type on line 6. When we enter numeric value 8, it is not stored as the value `8`, it is stored as `56` instead. When we enter numeric value 4, it is not stored as the value `4`, it is stored as `52` instead. The `readNumber()` function thus returns `56` and `52` instead of `8` and `4` as expected.

The solution is to change the data type on line 6 from `char` to `int`.

Author's note

It's hard to find good examples of simple programs that have non-obvious issues to debug, given the limited material covered so far. Any readers have any suggestions?

Finding issues before they become problems

When you make a semantic error, that error may or may not be immediately noticeable when you run your program. An issue may lurk undetected in your code for a long time before newly introduced code or changed circumstances cause it to manifest as a program malfunction. The longer an error sits in the code base before it is found, the more likely it is to be hard to find, and something that may have been easy to fix originally turns into a debugging adventure that eats up time and energy.

So what can we do about that?

Don't make errors

Well, the best thing is to not make errors in the first place. Here's a list of things that can help avoid making errors:

- Follow best practices.
- Don't program when tired or frustrated. Take a break and come back later.
- Understand where the common pitfalls are in a language (all those things we warn you not to do).
- Don't let your functions get too long.
- Prefer using the standard library to writing your own code, when possible.
- Comment your code liberally.
- Start with simple solutions, then layer in complexity incrementally.
- Avoid clever/non-obvious solutions.
- Optimize for readability and maintainability, not performance.

> Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?
>
> —Brian Kernighan, "The Elements of Programming Style", 2nd edition

Refactoring your code

As you add new capabilities to your programs ("behavioral changes"), you will find that some of your functions grow in length. As functions get longer, they get both more complex and harder to understand.

One way to address this is to break a single long function into multiple shorter functions. This process of making structural changes to your code without changing its behavior is called **refactoring**. The goal of refactoring is to make your program less complex by increasing its organization and modularity.

So how long is too long for a function? A function that takes up one vertical screen worth of code is generally regarded as too long – if you have to scroll to read the whole function, the function's comprehensibility drops significantly. Ideally, a function should be less than ten lines. Functions that are less than five lines are even better.

Remember that the goal here is to maximize comprehension and maintainability, not to minimize function length – abandoning best practices or using obscure coding techniques to save a line or two doesn't do your code any favors.

> **Key insight**
>
> When making changes to your code, make behavioral changes OR structural changes, and then retest for correctness. Making behavioral and structural changes at the same time tends to lead to more errors as well as errors that are harder to find.

An introduction to defensive programming

Errors can be not only of your own making (e.g. incorrect logic), but also occur when your users use the application in a way that you did not anticipate. For example, if you ask the user to enter an integer, and they enter a letter instead, how does your program behave in such a case? Unless you anticipated this, and added some error handling for this case, probably not very well.

Defensive programming is a practice whereby the programmer tries to anticipate all of the ways the software could be misused, either by

end-users, or by other developers (including the programmer themselves) using the code. These misuses can often be detected and then mitigated (e.g. by asking a user who entered bad input to try again).

We'll explore topics related to error handling in future lessons.

Finding errors fast

Since not making errors is difficult in large programs, the next best thing is to catch errors you do make quickly.

The best way to do this is to program a little bit at a time, and then test your code and make sure it works.

However, there are a few other techniques we can also use.

An introduction to testing functions

One common way to help uncover issues with your program is to write testing functions to "exercise" the code you've written. Here's a primitive attempt, more for illustrative purposes than anything:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

void testadd()
{
    std::cout << "This function should print: 2 0 0 -2\n";
    std::cout << add(1, 1) << ' ';
    std::cout << add(-1, 1) << ' ';
    std::cout << add(1, -1) << ' ';
    std::cout << add(-1, -1) << ' ';
}

int main()
{
    testadd();

    return 0;
}
```

The `testadd()` function tests the `add()` function by calling it with different values. If all the values match our expectations, then we can be reasonably confident the function works. Even better, we can keep this function around, and run it any time we change function `add` to ensure we haven't accidentally broken it.

This is a primitive form of **unit testing**, which is a software testing method by which small units of source code are tested to determine whether they are correct.

As with logging frameworks, there are many 3rd party unit testing frameworks that can be used. It's also possible to write your own, though we'll need more language features at our disposal to do the topic justice. We'll come back to some of this in a future lesson.

An introduction to constraints

Constraints-based techniques involve the addition of some extra code (that can be compiled out in a non-debug build, if desired) to check that some set of assumptions or expectations are not violated.

For example, if we were writing a function to calculate the factorial of a number, which expects a non-negative argument, the function

could check to make sure the caller passed in a non-negative number before proceeding. If the caller passes in a negative number, then the function could immediately error out rather than producing some indeterminate result, helping ensure the problem is caught immediately.

One common method of doing this is via *assert* and *static_assert*, which we cover in lesson 9.6 – Assert and static_assert.

Shotgunning for general issues

Programmers tend to make certain kinds of common mistakes, and some of those mistakes can be discovered by programs trained to look for them. These programs, generally known as **static analysis tools** (sometimes informally called *linters*) are programs that analyze your source code to identify specific semantic issues (in this context, *static* means that these tools analyze the source code without executing it). The issues found by static analysis tools may or may not be the cause of any particular problem you are having, but may help point out fragile areas of code or issues that can be problematic in certain circumstances.

You already have one static analysis tool at your disposal – your compiler! In addition to ensuring your program is syntactically correct, most modern C++ compilers will do some light static analysis to identify some common problems. For example, many compilers will warn you if you try to use a variable that has not been initialized. If you haven't already, turning up your compiler warning and error levels (see lesson 0.11 – Configuring your compiler: Warning and error levels) can help surface these.

Many static analysis tools exist, some of which can identify over 300 types of programming errors. On our small academic programs, use of a static analysis tool is optional, but using one may help you find areas where your code is non-compliant with best practices. On large programs, use of a static analysis tool is highly recommended, as it can surface tens or hundreds of potential issues.

> **Best practice**
>
> Use a static analysis tool on your programs to help find areas where your code is non-compliant with best practices.

> **For Visual Studio users**
>
> Visual Studio 2019 onward comes with a built-in static analysis tool. You can access it via *Build > Run Code Analysis on Solution (Alt+F11)*.

> **Tip**
>
> Some commonly recommended static analysis tools include:
>
> Free:
>
> - clang-tidy
> - cpplint
> - cppcheck (already integrated into Code::Blocks)
> - SonarLint
>
> Most of these have extensions that allow them to integrate into your IDE. For example, Clang Power Tools extension.
>
> Paid (may be free for Open Source projects):
>
> - Coverity
> - SonarQube

More debugging tactics

In the previous lesson (3.4 – Basic debugging tactics), we started exploring how to manually debug problems. In that lesson, we offered some criticisms of using statements to print debug text:

1. Debug statements clutter your code.
2. Debug statements clutter the output of your program.

3. Debug statements require modification of your code to both add and to remove, which can introduce new bugs.
4. Debug statements must be removed after you're done with them, which makes them non-reusable.

We can mitigate some of these issues. In this lesson, we'll explore some basic techniques for doing so.

Conditionalizing your debugging code

Consider the following program that contains some debug statements:

```
#include <iostream>

int getUserInput()
{
std::cerr << "getUserInput() called\n";
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
std::cerr << "main() called\n";
    int x{ getUserInput() };
    std::cout << "You entered: " << x << '\n';

    return 0;
}
```

When you're done with the debugging statement, you'll either need to remove them, or comment them out. Then if you want them again later, you'll have to add them back, or uncomment them.

One way to make it easier to disable and enable debugging throughout your program is to make your debugging statements conditional using preprocessor directives:

```
#include <iostream>

#define ENABLE_DEBUG // comment out to disable debugging

int getUserInput()
{
#ifdef ENABLE_DEBUG
std::cerr << "getUserInput() called\n";
#endif
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
#ifdef ENABLE_DEBUG
std::cerr << "main() called\n";
#endif
    int x{ getUserInput() };
    std::cout << "You entered: " << x << '\n';

    return 0;
}
```

Now we can enable debugging simply by commenting / uncommenting `#define ENABLE_DEBUG`. This allows us to reuse previously added debug statements and then just disable them when we're done with them, rather than having to actually remove them from the code. If this were a multi-file program, the `#define ENABLE_DEBUG` would go in a header file that's included into all code files so we can comment / uncomment the `#define` in a single location and have it propagate to all code files.

This addresses the issue with having to remove debug statements and the risk in doing so, but at the cost of even more code clutter. Another downside of this approach is that if you make a typo (e.g. misspell "DEBUG") or forget to include the header into a code file, some or all of the debugging for that file may not be enabled. So although this is better than the unconditionalized version, there's still room to improve.

Using a logger

An alternative approach to conditionalized debugging via the preprocessor is to send your debugging information to a log. A **log** is a sequential record of events that have happened, usually time-stamped. The process of generating a log is called **logging**. Typically, logs are written to a file on disk (called a **log file**) so they can be reviewed later. Most applications and operating systems write log files that can be used to help diagnose issues that occur.

Log files have a few advantages. Because the information written to a log file is separated from your program's output, you can avoid the clutter caused by mingling your normal output and debug output. Log files can also be easily sent to other people for diagnosis – so if someone using your software has an issue, you can ask them to send you the log file, and it might help give you a clue where the issue is.

C++ contains an output stream named `std::clog` that is intended to be used for writing logging information. However, by default, `std::clog` writes to the standard error stream (the same as `std::cerr`). And while you can redirect it to file instead, this is one area where you're generally better off using one of the many existing third-party logging tools available. Which one you use is up to you.

For illustrative purposes, we'll show what outputting to a logger looks like using the plog logger. Plog is implemented as a set of header files, so it's easy to include anywhere you need it, and it's lightweight and easy to use.

```
#include <plog/Log.h> // Step 1: include the logger headers
#include <plog/Initializers/RollingFileInitializer.h>
#include <iostream>

int getUserInput()
{
    PLOGD << "getUserInput() called"; // PLOGD is defined by the plog library

    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;
    return x;
}

int main()
{
    plog::init(plog::debug, "Logfile.txt"); // Step 2: initialize the logger

    PLOGD << "main() called"; // Step 3: Output to the log as if you were writing to the console

    int x{ getUserInput() };
    std::cout << "You entered: " << x << '\n';

    return 0;
}
```

Here's output from the above logger (in the `Logfile.txt` file):

```
2018-12-26 20:03:33.295 DEBUG [4752] [main@19] main() called
2018-12-26 20:03:33.296 DEBUG [4752] [getUserInput@7] getUserInput() called
```

How you include, initialize, and use a logger will vary depending on the specific logger you select.

Note that conditional compilation directives are also not required using this method, as most loggers have a method to reduce/eliminate writing output to the log. This makes the code a lot easier to read, as the conditional compilation lines add a lot of clutter. With plog, logging can be temporarily disabled by changing the init statement to the following:

```
plog::init(plog::none , "Logfile.txt"); // plog::none eliminates writing of most messages, essentially turning loggi
```

We won't use plog in any future lessons, so you don't need to worry about learning it.

> **As an aside…**
>
> If you want to compile the above example yourself, or use plog in your own projects, you can follow these instructions to install it:
>
> First, get the latest plog release:
>
> - Visit the plog repo.
> - Click the green Code button in the top right corner, and choose "Download zip"
>
> Next, unzip the entire archive to `somewhere` on your hard drive.
>
> Finally, for each project, set the `somewhere\plog-master\include\` directory as an `include directory` inside your IDE. There are instructions on how to do this for Visual Studio here: A.2 – Using libraries with Visual Studio and Code::Blocks here: A.3 – Using libraries with Code::Blocks. Since plog doesn't have a precompiled library file, you can skip the parts related to precompiled library files.

> The log file will generally be created in the same directory as your executable.

> **Tip**
>
> In larger or performance-sensitive projects, faster and more feature-rich loggers may be preferred, such as spdlog.

Syntax and Semantic Errors

Software errors are prevalent. It's easy to make them, and it's hard to find them. In this chapter, we'll explore topics related to the finding and removal of bugs within our C++ programs, including learning how to use the integrated debugger that is part of our IDE.

Although debugging tools and techniques aren't part of the C++ standard, learning to find and remove bugs in the programs you write is an extremely important part of being a successful programmer. Therefore, we'll spend a bit of time covering such topics, so that as the programs you write become more complex, your ability to diagnose and remedy issues advances at a similar pace.

If you have experience debugging programs in another programming language, much of this will be familiar to you.

Syntax errors

Programming can be challenging, and C++ is somewhat of a quirky language. Put those two together, and there are a lot of ways to make mistakes. Errors generally fall into one of two categories: syntax errors, and semantic errors (logic errors).

A **syntax error** occurs when you write a statement that is not valid according to the grammar of the C++ language. This includes errors such as missing semicolons, mismatched parentheses or braces, etc… For example, the following program contains quite a few syntax errors:

```
#include <iostream>

int main( // missing closing brace
{
    int 1x; // variable name can't start with number
    std::cout << "Hi there"; << x +++ << '\n'; // extraneous semicolon, operator+++ does not exist
    return 0 // missing semicolon at end of statement
}
```

Fortunately, the compiler will detect syntax errors and issue a compilation warning or error, so you easily identify and fix the problem. Then it's just a matter of compiling again until you get rid of all the errors.

Semantic errors

A **semantic error** is an error in meaning. These occur when a statement is syntactically valid, but either violates other rules of the language, or does not do what the programmer intended.

Some kind of semantic errors can be caught by the compiler. Common examples include using an undeclared variable, type mismatches (when we use an object with the wrong type somewhere), etc…

For example, the following program contains several compile-time semantic errors:

```
int main()
{
    5 = x; // x not declared, cannot assign a value to 5
    return "hello"; // "hello" cannot be converted to an int
}
```

Other semantic errors only manifest at runtime. Sometimes these will cause your program to crash, such as in the case of division by zero:

```
#include <iostream>

int main()
{
    int a { 10 };
    int b { 0 };
    std::cout << a << " / " << b << " = " << a / b << '\n'; // division by 0 is undefined in mathematics
    return 0;
}
```

More often these will just produce the wrong value or behavior:

```
#include <iostream>

int main()
{
    int x; // no initializer provided
    std::cout << x << '\n'; // Use of uninitialized variable leads to undefined result

    return 0;
}
```

or

```
#include <iostream>

int add(int x, int y) // this function is supposed to perform addition
{
    return x - y; // but it doesn't due to the wrong operator being used
}

int main()
{
    std::cout << "5 + 3 = " << add(5, 3) << '\n'; // should produce 8, but produces 2

    return 0;
}
```

or

```
#include <iostream>

int main()
{
    return 0; // function returns here

    std::cout << "Hello, world!\n"; // so this never executes
}
```

In the above example, the errors are fairly easy to spot. But in most non-trivial programs, runtime semantic errors are not easy to find by eyeballing the code. This is where debugging techniques can come in handy.

The Debugging Process

Let's say you've written a program, and it's not working correctly – the code compiles fine, but when you run it, you're getting an incorrect result. You must have a semantic error somewhere. How can you find it? If you've been following best practices by writing a little bit of code and then testing it, you may have a good idea where your error is. Or you may have no clue at all.

All bugs stem from a simple premise: Something that you thought was correct, isn't. Actually figuring out where that error is can be challenging. In this lesson, we'll outline the general process of debugging a program.

Because we haven't covered that many C++ topics yet, our example programs in this chapter are going to be pretty basic. That may make some of the techniques we're showing here seem excessive. However, keep in mind that these techniques are designed to be used with larger, more complex programs, and will be of more use in such a setting (which is where you need them most).

A general approach to debugging

Once a problem has been identified, debugging the problem generally consists of six steps:

1. Find the root cause of the problem (usually the line of code that's not working). We'll discuss some strategies on how to do this in the next lesson.
2. Ensure you understand why the issue is occurring.
3. Determine how you'll fix the issue.
4. Repair the issue causing the problem.
5. Retest to ensure the problem has been fixed.
6. Retest to ensure no new problems have emerged.

Let's use a real-life analogy here. Let's say one evening, you go to get some ice from the ice dispenser in your freezer. You put your cup up to the dispenser, press the lever, and … nothing comes out. Uh oh. You've discovered some kind of defect. What would you do? You'd probably start an investigation to see if you could identify the root cause of the issue.

Find the root cause: Since you hear the ice dispenser trying to deliver ice, it's probably not the ice delivery mechanism itself. So you open the freezer, and examine the ice tray. No ice. Is that the root cause of the issue? No, it's another symptom. After further examination, you determine that the ice maker does not appear to be making ice. Is the problem the ice maker or something else? The freezer is still cold, the water line isn't clogged, and everything else seems to be working, so you conclude that the root cause is that the ice maker is non-functional.

Understand the problem: This is simple in this case. A broken ice maker won't make ice.

Determine a fix: At this point, you have several options for a fix: You could work around the issue (buy bags of ice from the store). You could try to diagnose the ice-maker further, to see if there's a part that can be repaired. You could buy a new ice maker and install it in place of the current one. Or you could buy a new freezer. You decide to buy a new ice maker.

Repair the issue: Once the ice maker has arrived, you install it.

Retest: After turning the electricity back on and waiting overnight, your new ice maker starts making ice. No new issues are discovered.

Now let's apply this process to our simple program from the previous lesson:

```
#include <iostream>

int add(int x, int y) // this function is supposed to perform addition
{
    return x - y; // but it doesn't due to the wrong operator being used
}

int main()
{
    std::cout << "5 + 3 = " << add(5, 3) << '\n'; // should produce 8, but produces 2

    return 0;
}
```

This code is nice in one regard: the bug is very apparent, because the wrong answer gets printed to the screen via line 10. That gives us a starting point for our investigation.

Find the root cause: On line 10, we can see that we're passing in literals for arguments (5 and 3), so there is no room for error there. Since the inputs to function *add* are correct, but the output isn't, it's pretty apparent that function *add* must be producing the wrong value. The only statement in function *add* is the return statement, which must be the culprit. We've found the problem line. Now that we know where to focus our attention, noticing that we're subtracting instead of adding is something you're likely to find via inspection.

Understand the problem: In this case, it's obvious why the wrong value is being generated – we're using the wrong operator.

Determine a fix: We'll simply change *operator-* to *operator+*.

Repair the issue: This is actually changing *operator-* to *operator+* and ensuring the program recompiles.

Retest: After implementing the change, rerunning the program will indicate that our program now produces the correct value of 8. For this simple program, that's all the testing that's needed.

This example is trivial, but illustrates the basic process you'll go through when diagnosing any program.

Using an integrated debugger: Running and breakpoints

While stepping (covered in lesson 3.6 – Using an integrated debugger: Stepping) is useful for examining each individual line of your code in isolation, in a large program, it can take a long time to step through your code to even get to the point where you want to examine in more detail.

Fortunately, modern debuggers provide more tools to help us efficiently debug our programs. In this lesson, we'll look at some of the debugger features that let us more quickly navigate our code.

Run to cursor

The first useful command is commonly called *Run to cursor*. This **Run to cursor** command executes the program until execution reaches the statement selected by your cursor. Then it returns control to you so you can debug starting at that point. This makes for an efficient way to start debugging at a particular point in your code, or if already debugging, to move straight to some place you want to examine further.

> **For Visual Studio users**
>
> In Visual Studio, the *run to cursor* command can be accessed by right clicking a statement in your code and choosing *Run to Cursor* from the context menu, or by pressing the ctrl-F10 keyboard combo.

> **For Code::Blocks users**
>
> In Code::Blocks, the *run to cursor* command can be accessed by right clicking a statement in your code and choosing either *Run to cursor* from the context menu or *Debug menu > Run to cursor*, or by pressing the F4 shortcut key.

> **For VS Code users**
>
> In VS Code, the *run to cursor* command can be accessed while already debugging a program by right clicking a statement in your code and choosing *Run to Cursor* from the context menu.

Let's try it using the same program we've been using:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);

    return 0;
}
```

Simply right click anywhere on line 5, then choose "Run to cursor".

You will notice the program starts running, and the execution marker moves to the line you just selected. Your program has executed up to this point and is now waiting for your further debugging commands. From here, you can step through your program, *run to cursor* to a different location, etc…

If you *run to cursor* to a location that doesn't execute, *run to cursor* will simply run your program until termination.

Continue

Once you're in the middle of a debugging session, you may want to just run the program from that point forward. The easiest way to do this is to use the *continue* command. The **continue** debug command simply continues running the program as per normal, either until the program terminates, or until something triggers control to return back to you again (such as a breakpoint, which we'll cover later in this lesson).

> **For Visual Studio users**
>
> In Visual Studio, the *continue* command can be accessed while already debugging a program via *Debug menu > Continue*, or by pressing the F5 shortcut key.

> **For Code::Blocks users**
>
> In Code::Blocks, the *continue* command can be accessed while already debugging a program via *Debug menu > Start / Continue*, or by pressing the F8 shortcut key.

> **For VS Code users**
>
> In VS Code, the *continue* command can be accessed while already debugging a program via *Run menu > Continue*, or by pressing the *F5* shortcut key.

Let's test out the *continue* command. If your execution marker isn't already on line 5, *run to cursor* to line 5. Then choose *continue* from this point. Your program will finish executing and then terminate.

Start

The *continue* command has a twin brother named *start*. The *start* command performs the same action as *continue*, just starting from the beginning of the program. It can only be invoked when not already in a debug session.

> **For Visual Studio users**
>
> In Visual Studio, the *start* command can be accessed while not debugging a program via *Debug menu > Start Debugging*, or by pressing the F5 shortcut key.

> **For Code::Blocks users**
>
> In Code::Blocks, the *start* command can be accessed while not debugging a program via *Debug menu > Start / Continue*, or by pressing the F8 shortcut key.

> **For VS Code users**
>
> In VS Code, the *start* command can be accessed while not debugging a program via *Run menu > Start Debugging*, or by pressing the *F5* shortcut key.

If you use the *start* command on the above sample program, it will run all the way through without interruption (except on `VS Code`, because we set `stopAtEntry: true` in the prior lesson). While this may seem unremarkable, that's only because we haven't told the debugger to interrupt the program. We'll put this command to better use in the next section.

Breakpoints

The last topic we are going to talk about in this section is breakpoints. A **breakpoint** is a special marker that tells the debugger to stop execution of the program at the breakpoint when running in debug mode.

> **For Visual Studio users**
>
> In Visual Studio, you can set or remove a breakpoint via *Debug menu > Toggle Breakpoint*, or by right clicking on a statement and choosing *Toggle Breakpoint* from the context menu, or by pressing the F9 shortcut key, or by clicking to the left of the line number (in the light grey area).

> **For Code::Blocks users**
>
> In Code::Blocks, you can set or remove a breakpoint via *Debug menu > Toggle breakpoint*, or by right clicking on a statement and choosing *Toggle breakpoint* from the context menu, or by pressing the F5 shortcut key, or by clicking to the right of the line number.

> **For VS Code users**
>
> In VS Code, you can set or remove a breakpoint via *Run menu > Toggle Breakpoint*, or by pressing the *F9* shortcut key, or by clicking to the left of the line number.

When you set a breakpoint, you will see a new type of icon appear. Visual Studio uses a red circle, Code::Blocks uses a red octagon (like a stop sign):

Go ahead and set a breakpoint on the line 5, as shown in the image above.

Now, choose the *Start* command to have the debugger run your code, and let's see the breakpoint in action. You will notice that instead of running all the way to the end of the program, the debugger stops at the breakpoint (with the execution marker sitting on top of the breakpoint icon):

It's just as if you'd *run to cursor* to this point.

Breakpoints have a couple of advantages over *run to cursor*. First, a breakpoint will cause the debugger to return control to you every time they are encountered (unlike *run to cursor*, which only runs to the cursor once each time it is invoked). Second, you can set a breakpoint and it will persist until you remove it, whereas with *run to cursor* you have to locate the spot you want to run to each time you invoke the command.

Note that breakpoints placed on lines that are not in the path of execution will not cause the debugger to halt execution of the code.

Let's take a look at a slightly modified program that better illustrates the difference between breakpoints and *run to cursor*:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);
    printValue(6);
    printValue(7);

    return 0;
}
```

First, start a new debugging session and then do a *run to cursor* to line 5. Now choose *continue*. The program will continue to the end (it won't stop on line 5 again, even though line 5 is executed twice more).

Next, place a breakpoint on line 5, then choose *start*. The program will stop on line 5. Now choose *continue*. The program will stop on line 5 a second time. Choose *continue* again, and it will stop a third time. One more *continue*, and the program will terminate. You can see that

the breakpoint caused the program to stop as many times as that line was executed.

Set next statement

There's one more debugging command that's used fairly uncommonly, but is still at least worth knowing about, even if you won't use it very often. The **set next statement** command allows us to change the point of execution to some other statement (sometimes informally called *jumping*). This can be used to jump the point of execution forwards and skip some code that would otherwise execute, or backwards and have something that already executed run again.

> **For Visual Studio users**
>
> In Visual Studio, you can jump the point of execution by right clicking on a statement and choosing *Set next statement* from the context menu, or by pressing the Ctrl-Shift-F10 shortcut combination. This option is contextual and only occurs while already debugging a program.

> **For Code::Blocks users**
>
> In Code::Blocks, you can jump the point of execution via *Debug menu > Set next statement*, or by right clicking on a statement and choosing *Set next statement* from the context menu. Code::Blocks doesn't have a keyboard shortcut for this command.

> **For VS Code users**
>
> In VS Code, you can jump the point of execution by right clicking on a statement and choosing *Jump to cursor* from the context menu. This option is contextual and only occurs while already debugging a program.

Let's see jumping forwards in action:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);
    printValue(6);
    printValue(7);

    return 0;
}
```

First, *run to cursor* to line 11. At this point, you should see the value of *5* in the console output window.

Now, right click on line 12, and choose *set next statement*. This causes line 11 to be skipped and not execute. Then choose *continue* to finish executing your program.

The output of your program should look like this:

```
5
7
```

We can see that `printValue(6)` was skipped.

This functionality can be useful in several contexts.

In our exploration of basic debugging techniques, we discussed commenting out a function as a way to determine whether that function had a role in causing an issue. This requires modifying our code, and remembering to uncomment the function later. In the debugger, there's no direct way to skip a function, so if you decide you want to do this, using *set next statement* to jump over a function call is the easiest way to do so.

Jumping backwards can also be useful if we want to watch a function that just executed run again, so we can see what it is doing.

With the same code above, *run to cursor* to line 12. Then *set next statement* on line 11, and *continue*. The program's output should be:

```
5
6
6
7
```

> **Warning**
>
> The *set next statement* command will change the point of execution, but will not otherwise change the program state. Your variables will retain whatever values they had before the jump. As a result, jumping may cause your program to produce different values, results, or behaviors than it would otherwise. Use this capability judiciously (especially jumping backwards).

> **Warning**
>
> You should not use *set next statement* to change the point of execution to a different function. This may result in undefined behavior, and likely a crash.

"Step back" vs jumping backwards via "Set next statement"

"Step back" rewinds the state of everything, as if you'd never gone past that point in the first place. Any changes to variable values or other program state is undone. This is essentially an "undo" command for stepping.

"Set next statement" when used to jump backwards only changes the point of execution. Any changes to variable values or other program state are not undone.

Conclusion

You now learned the major ways that you can use an integrated debugger to watch and control how your program executes. While these commands can be useful for diagnosing code flow issues (e.g. to determine if certain functions are or aren't being called), they are only a portion of the benefit that the integrated debugger brings to the table. In the next lesson, we'll start exploring additional ways to examine your program's state, for which you'll need these commands as a prerequisite. Let's go!

Using an Integrated Debugger: Stepping

When you run your program, execution begins at the top of the *main* function, and then proceeds sequentially statement by statement, until the program ends. At any point in time while your program is running, the program is keeping track of a lot of things: the value of the variables you're using, which functions have been called (so that when those functions return, the program will know where to go back to), and the current point of execution within the program (so it knows which statement to execute next). All of this tracked information is called your **program state** (or just *state*, for short).

In previous lessons, we explored various ways to alter your code to help with debugging, including printing diagnostic information or using a logger. These are simple methods for examining the state of a program while it is running. Although these can be effective if used properly, they still have downsides: they require altering your code, which takes time and can introduce new bugs, and they clutter your code, making the existing code harder to understand.

Behind the techniques we've shown so far is an unstated assumption: that once we run the code, it will run to completion (only pausing to accept input) with no opportunity for us to intervene and inspect the results of the program at whatever point we want.

However, what if we were able to remove this assumption? Fortunately, most modern IDEs come with an integrated tool called a debugger that is designed to do exactly this.

The debugger

A **debugger** is a computer program that allows the programmer to control how another program executes and examine the program state while that program is running. For example, the programmer can use a debugger to execute a program line by line, examining the value of variables along the way. By comparing the actual value of variables to what is expected, or watching the path of execution through the code, the debugger can help immensely in tracking down semantic (logic) errors.

The power behind the debugger is twofold: the ability to precisely control execution of the program, and the ability to view (and modify, if desired) the program's state.

Initially, debuggers (such as gdb) were separate programs that had command-line interfaces, where the programmer had to type arcane commands to make them work. Later debuggers (such as early versions of Borland's turbo debugger) were still separate programs, but supplied a "graphical" front end to make working with them easier. These days, many modern IDEs have an **integrated debugger** – that is, a debugger that uses the same interface as the code editor, so you can debug using the same environment that you use to write your code (rather than having to switch programs).

While integrated debuggers are highly convenient and recommended for beginners, command line debuggers are well supported and still commonly used in environments that do not support graphical interfaces (e.g. embedded systems).

Nearly all modern debuggers contain the same standard set of basic features – however, there is little consistency in terms of how the menus to access these features are arranged, and even less consistency in the keyboard shortcuts. Although our examples will use screenshots from Microsoft Visual Studio (and we'll cover how to do everything in Code::Blocks as well), you should have little trouble figuring out how to access each feature we discuss no matter which IDE you are using.

> **Tip**
>
> Debugger keyboard shortcuts will only work if the IDE/integrated debugger is the active window.

The remainder of this chapter will be spent learning how to use the debugger.

> **Tip**
>
> Don't neglect learning to use a debugger. As your programs get more complicated, the amount of time you spend learning to use the integrated debugger effectively will pale in comparison to amount of time you save finding and fixing issues.

> **Warning**
>
> Before proceeding with this lesson (and subsequent lessons related to using a debugger), make sure your project is compiled using a debug build configuration (see 0.9 – Configuring your compiler: Build configurations for more information).

> If you're compiling your project using a release configuration instead, the functionality of the debugger may not work correctly (e.g. when you try to step into your program, it will just run the program instead).

> **For Code::Blocks users**
>
> If you're using Code::Blocks, your debugger may or may not be set up correctly. Let's check.
>
> First, go to *Settings menu > Debugger…*. Next, open the *GDB/CDB debugger* tree on the left, and choose *Default*. A dialog should open that looks something like this:
>
> If you see a big red bar where the "Executable path" should be, then you need to locate your debugger. To do so, click the … button to the right of the *Executable path* field. Next, find the "gdb32.exe" file on your system – mine was in *C:Files (x86)32.exe*. Then click *OK*.

> **For Code::Blocks users**
>
> There have been reports that the Code::Blocks integrated debugger (GDB) can have issues recognizing some file paths that contain spaces or non-English characters in them. If the debugger appears to be malfunctioning as you go through these lessons, that could be a reason why.

> **For VS Code users**
>
> To set up debugging, press *Ctrl+Shift+P* and select "C/C++: Add Debug Configuration", followed by "C/C++: g++ build and debug active file". This should create and open the `launch.json` configuration file. Change the "stopAtEntry" to true:
>
> `"stopAtEntry": true,`
>
> Then open *main.cpp* and start debugging by pressing *F5* or by pressing *Ctrl+Shift+P* and selecting "Debug: Start Debugging and Stop on Entry".

Stepping

We're going to start our exploration of the debugger by first examining some of the debugging tools that allow us to control the way a program executes.

Stepping is the name for a set of related debugger features that let us execute (step through) our code statement by statement.

There are a number of related stepping commands that we'll cover in turn.

Step into

The **step into** command executes the next statement in the normal execution path of the program, and then pauses execution of the program so we can examine the program's state using the debugger. If the statement being executed contains a function call, *step into* causes the program to jump to the top of the function being called, where it will pause.

Let's take a look at a very simple program:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);

    return 0;
}
```

Let's debug this program using the *step into* command.

First, locate and then execute the *step into* debug command once.

> **For Visual Studio users**
>
> In Visual Studio, the *step into* command can be accessed via *Debug menu > Step Into*, or by pressing the F11 shortcut key.

> **For Code::Blocks users**
>
> In Code::Blocks, the *step into* command can be accessed via *Debug menu > Step into*, or by pressing the Shift-F7 shortcut combo.

> **For VS Code users**
>
> In VS Code, the *step into* command can be accessed via *Run > Step Into*.

> **For other compilers / IDEs**
>
> If using a different IDE, you'll likely find the *step into* command under a Debug or Run menu.

When your program isn't running and you execute the first debug command, you may see quite a few things happen:

- The program will recompile if needed.
- The program will begin to run. Because our application is a console program, a console output window should open. It will be empty because we haven't output anything yet.
- Your IDE may open some diagnostic windows, which may have names such as "Diagnostic Tools", "Call Stack", and "Watch". We'll cover what some of these are later – for now you can ignore them.

Because we did a *step into*, you should now see some kind of marker appear to the left of the opening brace of function *main* (line 9). In Visual Studio, this marker is a yellow arrow (Code::Blocks uses a yellow triangle). If you are using a different IDE, you should see something that serves the same purpose.

This arrow marker indicates that the line being pointed to will be executed next. In this case, the debugger is telling us that the next line to be executed is the opening brace of function *main* (line 9).

Choose *step into* (using the appropriate command for your IDE, listed above) to execute the opening brace, and the arrow will move to the next statement (line 10).

This means the next line that will be executed is the call to function *printValue*.

Choose *step into* again. Because this statement contains a function call to *printValue*, we step into the function, and the arrow will move to the top of the body of *printValue* (line 4).

Choose *step into* again to execute the opening brace of function *printValue*, which will advance the arrow to line 5.

Choose *step into* yet again, which will execute the statement `std::cout << value << '\n'` and move the arrow to line 6.

> **Warning**
>
> The version of operator<< used for output is implemented as a function. As a result, your IDE may step into the implementation of the operator<< function instead.
>
> If this happens, you'll see your IDE open a new code file, and the arrow marker will move to the top of a function named operator<< (this is part of the standard library). Close the code file that just opened, then find and execute *step out* debug command (instructions are below under the "step out" section, if you need help).

Now because `std::cout << value << '\n'` has executed, we should see the value *5* appear in the console window.

> **Tip**
>
> In a prior lesson, we mentioned that std::cout is buffered, which means there may be a delay between when you ask std::cout to print a value, and when it actually does. Because of this, you may not see the value 5 appear at this point. To ensure that all output from std::cout is output immediately, you can temporarily add the following statement to the top of your main() function:
>
> ```
> std::cout << std::unitbuf; // enable automatic flushing for std::cout (for debugging)
> ```
>
> For performance reasons, this statement should be removed or commented out after debugging.
>
> If you don't want to continually add/remove/comment/uncomment the above, you can wrap the statement in a conditional compilation preprocessor directive (covered in lesson 2.10 – Introduction to the preprocessor):
>
> ```
> #ifdef DEBUG
> std::cout << std::unitbuf; // enable automatic flushing for std::cout (for debugging)
> #endif
> ```
>
> You'll need to make sure the DEBUG preprocessor macro is defined, either somewhere above this statement, or as part of your compiler settings.

Choose *step into* again to execute the closing brace of function *printValue*. At this point, *printValue* has finished executing and control is returned to *main*.

You will note that the arrow is again pointing to *printValue*!

While you might think that the debugger intends to call *printValue* again, in actuality the debugger is just letting you know that it is returning from the function call.

Choose *step into* three more times. At this point, we have executed all the lines in our program, so we are done. Some debuggers will terminate the debugging session automatically at this point, others may not. If your debugger does not, you may need to find a "Stop Debugging" command in your menus (in Visual Studio, this is under *Debug > Stop Debugging*).

Note that *Stop Debugging* can be used at any point in the debugging process to end the debugging session.

Congratulations, you've now stepped through a program and watched every line execute!

> **Tip**
>
> In future lessons, we'll explore other debugger commands, some of which may not be available unless the debugger is already running. If the desired debugging command is not available, *step into* your code to start the debugger and try again.

Step over

Like *step into*, The **step over** command executes the next statement in the normal execution path of the program. However, whereas *step into* will enter function calls and execute them line by line, *step over* will execute an entire function without stopping and return control to

you after the function has been executed.

> **For Visual Studio users**
>
> In Visual Studio, the *step over* command can be accessed via *Debug menu > Step Over*, or by pressing the F10 shortcut key.

> **For Code::Blocks users**
>
> In Code::Blocks, the *step over* command is called *Next line* instead, and can be accessed via *Debug menu > Next line*, or by pressing the F7 shortcut key.

> **For VS Code users**
>
> In VS Code, the *step over* command can be accessed via *Run > Step Over*, or by pressing the *F10* shortcut key.

Let's take a look at an example where we step over the function call to *printValue*:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);

    return 0;
}
```

First, use *step into* on your program until the execution marker is on line 10:

Now, choose *step over*. The debugger will execute the function (which prints the value *5* in the console output window) and then return control to you on the next statement (line 12).

The *step over* command provides a convenient way to skip functions when you are sure they already work or are not interested in debugging them right now.

Step out

Unlike the other two stepping commands, **Step out** does not just execute the next line of code. Instead, it executes all remaining code in the function currently being executed, and then returns control to you when the function has returned.

> **For Visual Studio users**
>
> In Visual Studio, the *step out* command can be accessed via *Debug menu > Step Out*, or by pressing the Shift-F11 shortcut combo.

> **For Code::Blocks users**
>
> In Code::Blocks, the *step out* command can be accessed via *Debug menu > Step out*, or by pressing the ctrl-F7 shortcut combo.

> **For VS Code users**
>
> In VS Code, the *step out* command can be accessed via *Run > Step Out*, or by pressing the *shift+F11* shortcut combo.

Let's take a look at an example of this using the same program as above:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value << '\n';
}

int main()
{
    printValue(5);

    return 0;
}
```

Step into the program until you are inside function *printValue*, with the execution marker on line 4.

Then choose *step out*. You will notice the value *5* appears in the output window, and the debugger returns control to you after the function has terminated (on line 10).

This command is most useful when you've accidentally stepped into a function that you don't want to debug.

A step too far

When stepping through a program, you can normally only step forward. It's very easy to accidentally step past (overstep) the place you wanted to examine.

If you step past your intended destination, the usual thing to do is stop debugging and restart debugging again, being a little more careful not to pass your target this time.

Step back

Some debuggers (such as Visual Studio Enterprise Edition and rr) have introduced a stepping capability generally referred to as *step back* or *reverse debugging*. The goal of a *step back* is to rewind the last step, so you can return the program to a prior state. This can be useful if you overstep, or if you want to re-examine a statement that just executed.

Implementing *step back* requires a great deal of sophistication on the part of the debugger (because it has to keep track of a separate program state for each step). Because of the complexity, this capability isn't standardized yet, and varies by debugger. As of the time of writing (Jan 2019), neither Visual Studio Community edition nor the latest version of Code::Blocks support this capability. Hopefully at some point in the future, it will trickle down into these products and be available for wider use.

Using an integrated debugger: The call stack

Modern debuggers contain one more debugging information window that can be very useful in debugging your program, and that is the call stack window.

When your program calls a function, you already know that it bookmarks the current location, makes the function call, and then returns. How does it know where to return to? The answer is that it keeps track in the call stack.

The **call stack** is a list of all the active functions that have been called to get to the current point of execution. The call stack includes an entry for each function called, as well as which line of code will be returned to when the function returns. Whenever a new function is called, that function is added to the top of the call stack. When the current function returns to the caller, it is removed from the top of the call stack, and control returns to the function just below it.

The **call stack window** is a debugger window that shows the current call stack. If you don't see the call stack window, you will need to tell the IDE to show it.

> **For Visual Studio users**
>
> In Visual Studio, the call stack window can be found via *Debug menu > Windows > Call Stack*. Note that you have to be in a debug session to activate this window.

> **For Code::Blocks users**
>
> In Code::Blocks, the call stack window can be found via *Debug menu > Debugging windows > Call stack*.

> **For VS Code users**
>
> In VS Code, the call stack window appears in debug mode, docked on the left.

Let's take a look at the call stack using a sample program:

```
#include <iostream>

void a()
{
    std::cout << "a() called\n";
}

void b()
{
    std::cout << "b() called\n";
    a();
}

int main()
{
    a();
    b();

    return 0;
}
```

Put breakpoints on lines 5 and 10 of this program, and then start debugging mode. Because function *a* is called first, the breakpoint on line 5 will be hit first.

At this point, you should see something like this:

Your IDE may exhibit some differences:

- The format of your function names and line numbers may be different
- Your line numbers may be slightly different (off by 1)
- Instead of *[External Code]* you may see a bunch of other crazily named functions.

These differences are inconsequential.

What's relevant here is the top two lines. From the bottom up, we can see that function *main* was called first, and then that function *a* was called next.

The *line 5* next to function *a* shows us where the current point of execution is (which matches the execution marker in the code window). The *line 17* on the second line indicates the line that will be returned to when control returns to function *main*.

> **Tip**

> The line numbers after the function names show the next line to be executed in each function.
>
> Since the top entry on the call stack represents the currently executing function, the line number here shows the next line that will execute when execution resumes. The remaining entries in the call stack represent functions that will be returned to at some point, so the line number for these represent the next statement that will execute after the function is returned to.

Now, choose the *continue* debug command to advance execution to the next breakpoint, which will be on line 10. The call stack should update to reflect the new situation:

You'll notice that function *b* is now the top line of the call stack, reflecting the fact that function *b* is the function that is actively being executed. Note that function *a* is no longer represented on the call stack. This is because function *a* was removed from the call stack when it returned.

Choose the *continue* debug command one more time, and we'll hit the breakpoint on line 5 again (because function *b* calls function *a*). The call stack will look like this:

There are now three functions on the call stack: (from bottom to top) *main*, which called function *b*, which called function *a*.

The call stack is useful in conjunction with breakpoints, when your breakpoint is hit and you want to know what functions were called to get to that specific point in the code.

Conclusion

Congratulations, you now know the basics of using an integrated debugger! Using stepping, breakpoints, watches, and the call stack window, you now have the fundamentals to be able to debug almost any problem. Like many things, becoming good at using a debugger takes some practice and some trial and error. But again, we'll reiterate the point that the time devoted to learning how to use an integrated debugger effectively will be repaid many times over in time saved debugging your programs!

Using an Integrated Debugger: Watching Variables

In the previous lessons (3.6 – Using an integrated debugger: Stepping and 3.7 – Using an integrated debugger: Running and breakpoints), you learned how to use the debugger to watch the path of execution through your program. However, stepping through a program is only half of what makes the debugger useful. The debugger also lets you examine the value of variables as you step through your code, all without having to modify your code.

As per previous lessons, our examples here will use Visual Studio – if you are using a different IDE/debugger, the commands may have slightly different names or be located in different locations.

> **Warning**
>
> In case you are returning, make sure your project is compiled using a debug build configuration (see 0.9 – Configuring your compiler: Build configurations for more information). If you're compiling your project using a release configuration instead, the functionality of the debugger may not work correctly.

Watching variables

Watching a variable is the process of inspecting the value of a variable while the program is executing in debug mode. Most debuggers provide several ways to do this.

Let's take a look at a sample program:

```
#include <iostream>

int main()
{
    int x{ 1 };
    std::cout << x << ' ';

    x = x + 2;
    std::cout << x << ' ';

    x = x + 3;
    std::cout << x << ' ';

    return 0;
}
```

This is a pretty straightforward sample program – it prints the numbers 1, 3, and 6.

First, *run to cursor* to line 6.

At this point, the variable x has already been created and initialized with the value 1, so when we examine the value of x, we should expect to see the value 1.

The easiest way to examine the value of a simple variable like x is to hover your mouse over the variable x. Some modern debuggers support this method of inspecting simple variables, and it is the most straightforward way to do so.

> **For Code::Blocks users**
>
> If you're using Code::Blocks, this option is (inexplicably) off by default. Let's turn it on. First, go to *Settings menu > Debugger…*. Then under the *GDB/CDB debugger node*, select the *Default* profile. Finally, check the box labeled *Evaluate expression under cursor*.

Hover your mouse cursor over variable x on line 6, and you should see something like this:

Note that you can hover over any variable x, not just the one on the current line. For example, if we hover over the x on line 12, we'll see

the same value:

If you're using Visual Studio, you can also use QuickWatch. Highlight the variable name x with your mouse, and then choose "QuickWatch" from the right-click menu.

This will pull up a subwindow containing the current value of the variable:

Go ahead and close QuickWatch if you opened it.

Now let's watch this variable change as we step through the program. Either choose *step over* twice, or *run to cursor* to line 9. The variable x should now have value *3*. Inspect it and make sure that it does!

The watch window

Using the mouse hover or QuickWatch methods to inspect variables is fine if you want to know the value of a variable at a particular point in time, but it's not particularly well suited to watching the value of a variable change as you run the code because you continually have to rehover/reselect the variable.

In order to address this issue, all modern integrated debuggers provide another feature, called a watch window. The **watch window** is a window where you can add variables you would like to continually inspect, and these variables will be updated as you step through your program. The watch window may already be on your screen when you enter debug mode, but if it is not, you can bring it up through your IDE's window commands (these are typically found in a View or Debug menu).

> **For Visual Studio users**
>
> In Visual Studio, the watch window can be found at *Debug menu > Windows > Watch > Watch 1*. Do note that you have to be in debug mode for this option to be enabled, so *step into* your program first.
>
> Where this window appears (docked left, right, or bottom) may vary. You can change where it is docked by dragging the *Watch 1* tab to a different side of the application window.

> **For Code::Blocks users**
>
> In Code::Blocks, the watch window can be found at *Debug menu > Debugging windows > Watches*. This window will likely appear as a separate window. You can dock it into your main window by dragging it over.

> **For VS Code users**
>
> In VS Code, the watch window appears in debug mode, docked on the left above the call stack.

You should now see something like this:

The watches window may or may not contain anything in it already.

There are typically two different ways to add variables to the watch window:

1. Pull up the watch window, and type in the name of the variable you would like to watch in the leftmost column of the watch window.
2. In the code window, right click on the variable you'd like to watch, and choose *Add Watch* (Visual Studio) or *Watch x* (replace x with the variable's name) (Code::Blocks).

If you're not already in a debugging session with the execution marker on line 9 of your program, start a new debugging session and *run to cursor* to line 9.

Now, go ahead and add the variable "x" to your watch list. You should now see this:

Now *step over* twice, or *run to cursor* to line 12, and you should see the value of *x* change from *3* to *6*.

Variables that go out of scope (e.g. a local variable inside a function that has already returned to the caller) will stay in your watch window, but will generally either be marked as "not available", or may show the last known value but grayed out. If the variable returns to scope (e.g. the function is called again), its value will begin showing again. Therefore, it's fine to leave variables in the watch window, even if they're out of scope.

Using watches is the best way to watch the value of a variable change over time as you step through your program.

Setting a breakpoint on watched variables

Some debuggers will allow you to set a breakpoint on a watched variable rather than a line. This will cause the program to stop execution whenever the value of that variable changes.

For example, setting such a breakpoint on variable `x` in the above program will cause the debugger to stop after executing lines 8 and 11 (which is where the value of `x` is changed).

> **For Visual Studio users**
>
> In Visual Studio, make sure your variable is being watched. Next, *step into* your program and go to the watch window. Right click on the variable and select "Break when value changes".
>
> You will need to re-enable "Break when value changes" each time you start a debugging session.

The watch window can evaluate expressions too

The watch window will also allow you to evaluate simple expressions. If you haven't already, *run to cursor* to line 12. Then try entering *x + 2* into the watch window and see what happens (it should evaluate to 8).

You can also highlight an expression in your code and then inspect the value of that expression via hover or by adding it to the watch window via the right-click context menu.

> **Warning**
>
> Identifiers in watched expressions will evaluate to their current values. If you want to know what value an expression in your code is actually evaluating to, *run to cursor* to it first, so that all identifiers have the correct values.

Local watches

Because inspecting the value of local variables inside a function is common while debugging, many debuggers will offer some way to quickly watch the value of *all* local variables in scope.

> **For Visual Studio users**
>
> In Visual Studio, you can see the value of all local variables in the *Locals* window, which can be found at *Debug menu > Windows > Locals*. Note that you have to be in a debug session to activate this window.

> **For Code::Blocks users**
>
> In Code::Blocks, this is integrated into the *Watch* window, under the *Locals* node. If you don't see any, there either aren't any, or you need to uncollapse the node.

> **For VS Code users**
>
> In VS Code, the value of local variables can be found in the *VARIABLES* section that appears docked to the left in debug mode. You may need to uncollapse the *Locals* node.

If you're just looking to watch the value of a local variable, check the *locals* window first. It should already be there.